

► PySpark DataFrames

1. How do you handle schema inference and manual schema definition in DataFrames?

Ans:

Schema inference: PySpark automatically infers the schema when reading data. For example, when loading a CSV file:

```
df = spark.read.csv("file.csv", header=True, inferSchema=True)
```

Manual schema definition: You can define the schema using StructType and StructField when reading data:

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

schema = StructType([
    StructField("name", StringType(), True),
    StructField("age", IntegerType(), True)
])

df = spark.read.csv("file.csv", schema=schema)
```

2. Write a PySpark script to find duplicate records in a DataFrame?

Ans:

```
df.groupBy("column1", "column2").count().filter("count > 1").show()
```

3. Explain the concept of lazy evaluation with an example in PySpark DataFrames?

Ans:

Lazy evaluation means Spark does not execute operations immediately; instead, it builds a logical plan of transformations. The execution happens only when an action (like `.show()` or `.collect()`) is called.

```
df = spark.read.csv("file.csv")

transformed_df = df.filter("age > 30").select("name")

# No execution happens here, only when we do:

transformed_df.show() # Execution occurs now
```

4. How would you repartition or coalesce a DataFrame to optimize performance?

Ans:

Repartition: Used to increase or decrease the number of partitions. It involves a full shuffle.

```
df_repartitioned = df.repartition(5) # Increase partitions to 5
```

Coalesce: Merges the partitions into fewer ones without a full shuffle, and is more efficient when reducing partitions.

```
df_coalesced = df.coalesce(2) # Reduce partitions to 2\
```

5. Demonstrate how to use selectExpr to perform SQL-like operations on a DataFrame?

Ans:

```
df.selectExpr("age + 1 as new_age", "name").show()
```

► PySpark RDDs

1. What are the main transformations and actions in RDDs, and how are they different?

Ans:

Transformations: Lazy operations that return a new RDD (e.g., map, filter, flatMap, reduceByKey).

Actions: Trigger execution of transformations and return results (e.g., collect, count, first, reduce).

2. Show how to implement a word count using PySpark RDDs?

Ans:

```
rdd = sc.textFile("text.txt")
```

```
words = rdd.flatMap(lambda line: line.split(" "))
```

```
word_counts = words.map(lambda word: (word, 1)).reduceByKey(lambda a, b: a + b)
```

```
word_counts.collect()
```

3. What is the role of persist() and cache() in RDDs? Provide examples?

Ans:

persist(): Stores RDDs in memory or disk based on the storage level specified.

cache(): A shorthand for persist() with the default storage level (memory only).

```
rdd.persist(StorageLevel.MEMORY_AND_DISK)
```

```
rdd.cache() # Same as rdd.persist(StorageLevel.MEMORY_ONLY)
```

4. How do you filter RDD elements based on a custom condition in PySpark?

Ans:

```
rdd = sc.parallelize([1, 2, 3, 4, 5])  
  
filtered_rdd = rdd.filter(lambda x: x % 2 == 0) # Filter even numbers  
  
filtered_rdd.collect() # Output: [2, 4]
```

5. Compare RDDs with DataFrames. When would you choose one over the other?

Ans:

RDD: Low-level abstraction, more control, but less optimization. Use when fine-grained control over data is needed.

DataFrame: Higher-level abstraction, optimized using Catalyst and Tungsten. Prefer DataFrames for performance, ease of use, and SQL compatibility.

► Spark SQL

1. How can you register a DataFrame as a temporary SQL table in PySpark?

Ans:

```
df.createOrReplaceTempView("temp_table")
```

2. Write a SQL query in PySpark to find customers with the highest purchases?

Ans:

```
spark.sql("""  
    SELECT customer_id, SUM(purchase_amount) AS total_purchase  
    FROM purchases  
    GROUP BY customer_id  
    ORDER BY total_purchase DESC  
    LIMIT 1  
""").show()
```

3. How would you join two tables in PySpark SQL and filter the results based on conditions?

Ans:

```
df1 = spark.read.csv("table1.csv")  
df2 = spark.read.csv("table2.csv")  
df1.createOrReplaceTempView("table1")  
df2.createOrReplaceTempView("table2")  
spark.sql("""
```

```
SELECT t1.*, t2.*  
FROM table1 t1  
JOIN table2 t2 ON t1.id = t2.id  
WHERE t1.age > 30  
""").show()
```

4. Discuss the usage of window functions in PySpark SQL. Provide a sample query?

Ans:

```
from pyspark.sql.window import Window  
  
windowSpec = Window.partitionBy("customer_id").orderBy("purchase_date")  
  
df.withColumn("rank", rank().over(windowSpec)).show()
```

5. What is the difference between groupBy and groupByKey in PySpark?

Ans:

groupBy: Groups DataFrame rows based on columns (SQL-style aggregation).

groupByKey: Operates on RDDs and groups the data by key (typically used for key-value pair RDDs).

► PySpark Streaming

1. How does PySpark handle real-time data processing in structured streaming?

Ans:

PySpark Structured Streaming processes data in small batches or continuously using triggers. It automatically manages the ingestion, transformation, and output of streaming data.

```
df = spark.readStream.format("kafka").option("subscribe", "topic_name").load()
```

2. Explain the concept of watermarking in PySpark streaming with an example.

Ans:

Watermarking allows you to handle late-arriving data by setting a threshold on event time.

```
df.withWatermark("event_time", "10 minutes").groupBy("key").agg(func.count("value"))
```

3. Write a PySpark code to process a Kafka stream and store the results in a database.

Ans:

```
df = spark.readStream.format("kafka").option("kafka.bootstrap.servers",
"localhost:9092").option("subscribe", "topic").load()

# Process and write to a database

df.writeStream.format("jdbc").option("url", "jdbc:mysql://localhost:3306/db").option("dbtable",
"processed_data").start()
```

4. How do you handle late-arriving data in PySpark structured streaming?

Ans:

Use watermarking to delay the processing of late data by specifying a time threshold.

```
df.withWatermark("event_time", "10 minutes").groupBy("key").agg(func.count("value"))
```

5. Describe the role of triggers in PySpark structured streaming applications.

Ans:

Triggers control when the query execution happens in a stream (e.g., once per batch, continuous).

```
df.writeStream.trigger(processingTime="10 seconds").start()
```

► Performance Optimization

1. How can you optimize PySpark jobs using partitioning and bucketing?

Ans:

Partitioning: Split data into multiple partitions for parallel processing.

```
df.repartition(10)
```

Bucketing: Predefined division of data into buckets based on a column value.

```
df.write.bucketBy(4, "column_name").saveAsTable("bucketed_table")
```

2. Discuss the importance of broadcast variables in PySpark, and provide an example.

Ans:

Broadcast variables are used to send large read-only data to all workers efficiently. Example:

```
broadcast_var = sc.broadcast([1, 2, 3])
```

```
rdd = rdd.map(lambda x: x + broadcast_var.value[0])
```

3. Explain the Catalyst optimizer in PySpark and how it helps in query planning.

Ans:

- Catalyst is an optimization framework in Spark SQL that automatically optimizes query execution plans, such as predicate pushdown, constant folding, and query rewriting.

4. How would you profile and debug a PySpark application to identify bottlenecks?

Ans:

Use Spark UI to examine job stages, tasks, and shuffles. Use `SparkContext.setLogLevel("DEBUG")` for more detailed logs.

5. What are the best practices for managing shuffle operations in PySpark jobs?

Ans:

Minimize shuffle by reducing data movement, using appropriate partitioning strategies (repartition or coalesce), and ensuring data is evenly distributed across partitions.

---created by [Ahmed Mohiuddin | LinkedIn](#)